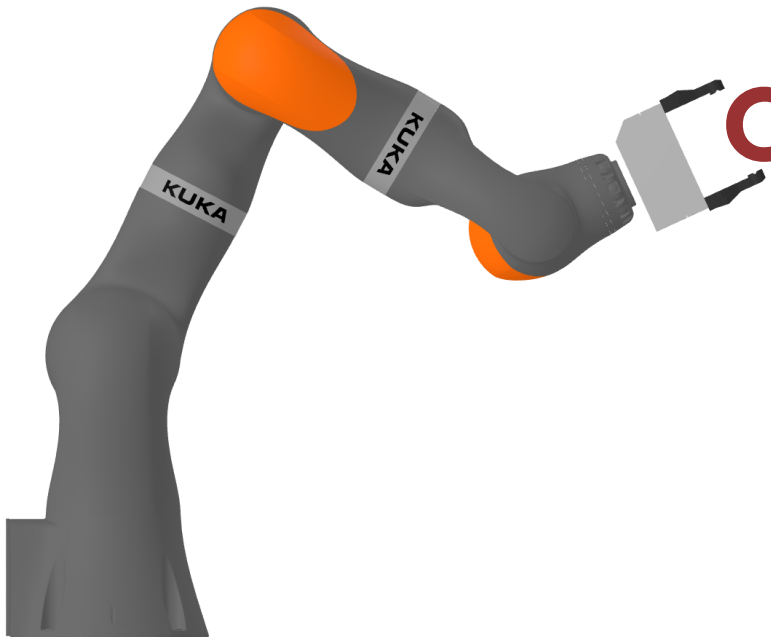


Geometric perception

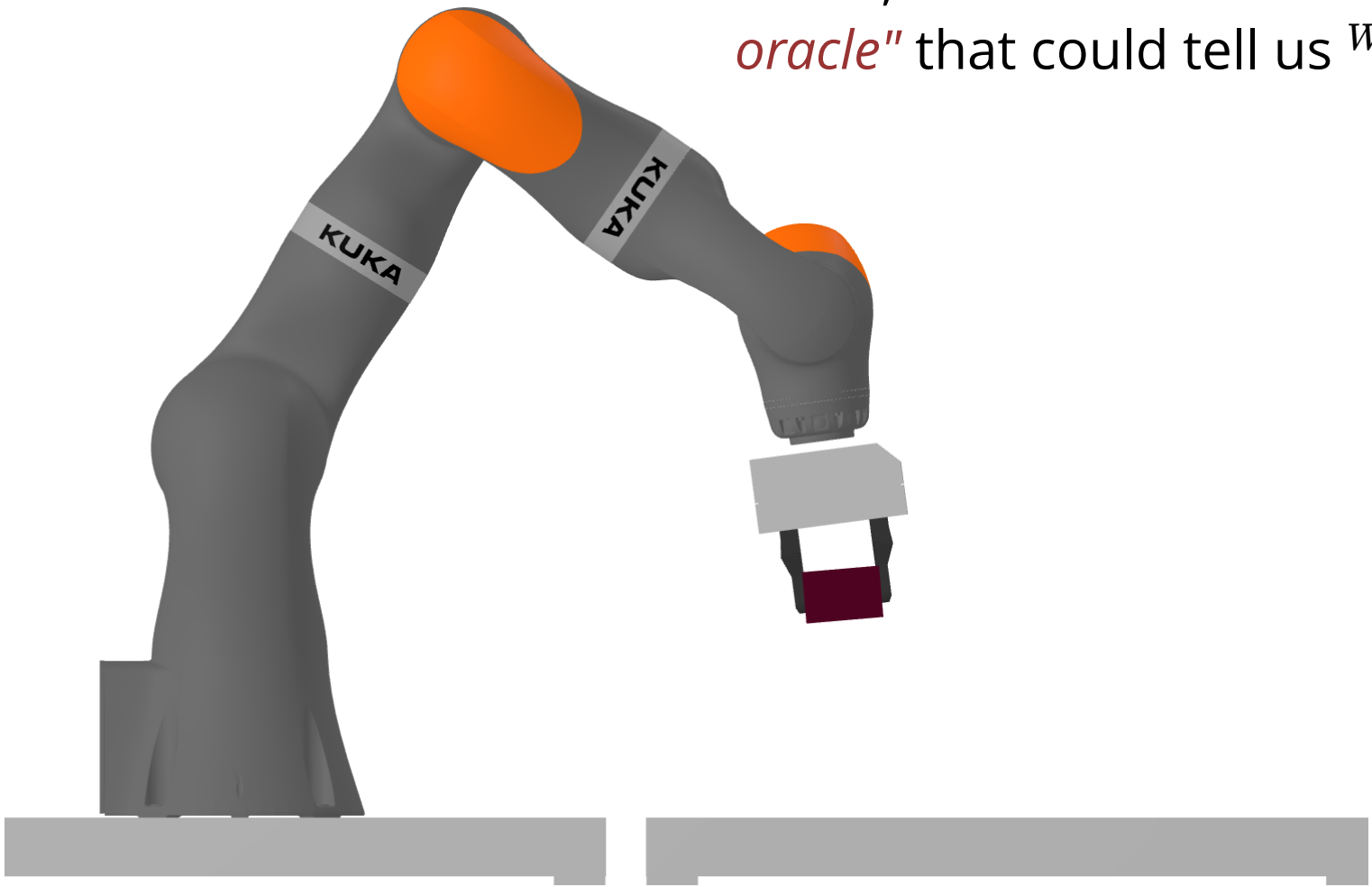
MIT 6.4210/2:

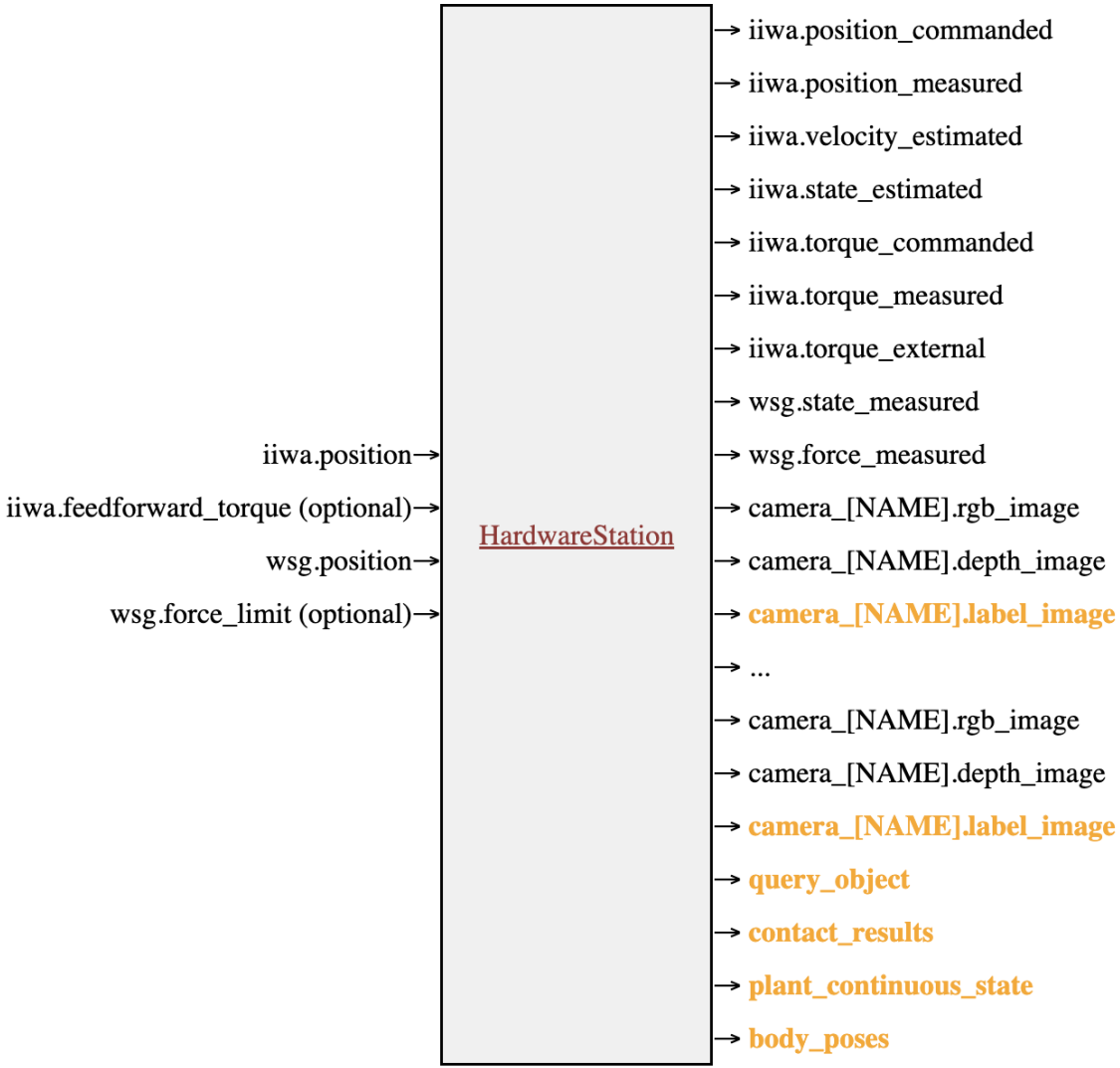
Robotic Manipulation

Fall 2025, Lecture 6

Massachusetts Institute of Technology

So far, we have assumed a *"perception oracle"* that could tell us ${}^{W}X^{O}$.





Use the cameras instead!

Stop using "cheat ports"

Pick and Place with Perception

https://people.csail.mit.edu/russt/uploads/pose_chapter_putting_it_all_together.html

Geometry sensors -- lidar / time of flight



Velodyne spinning lidar



Luminar

(500m range)



Hokoyu scanning lidar

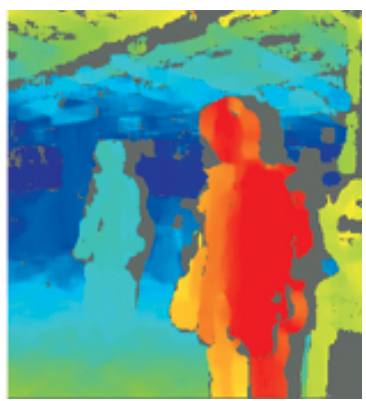


Geometry sensors -- stereo

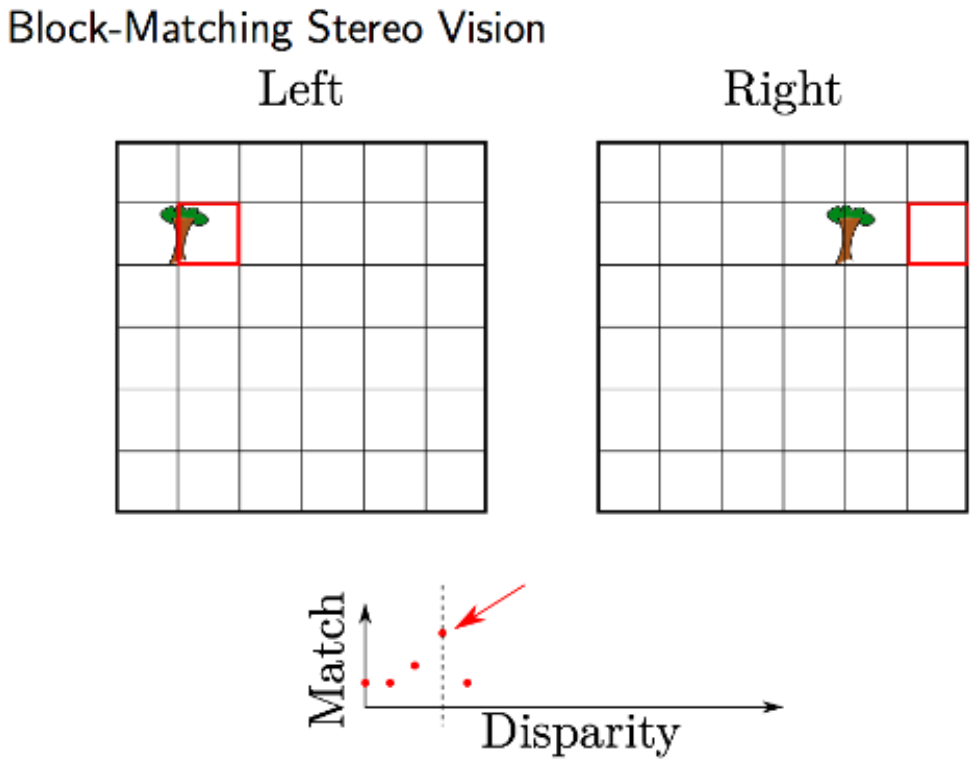


Carnegie multisense stereo





Point Grey Bumblebee



Geometry sensors -- structured light



Microsoft Kinect



Asus Xtion

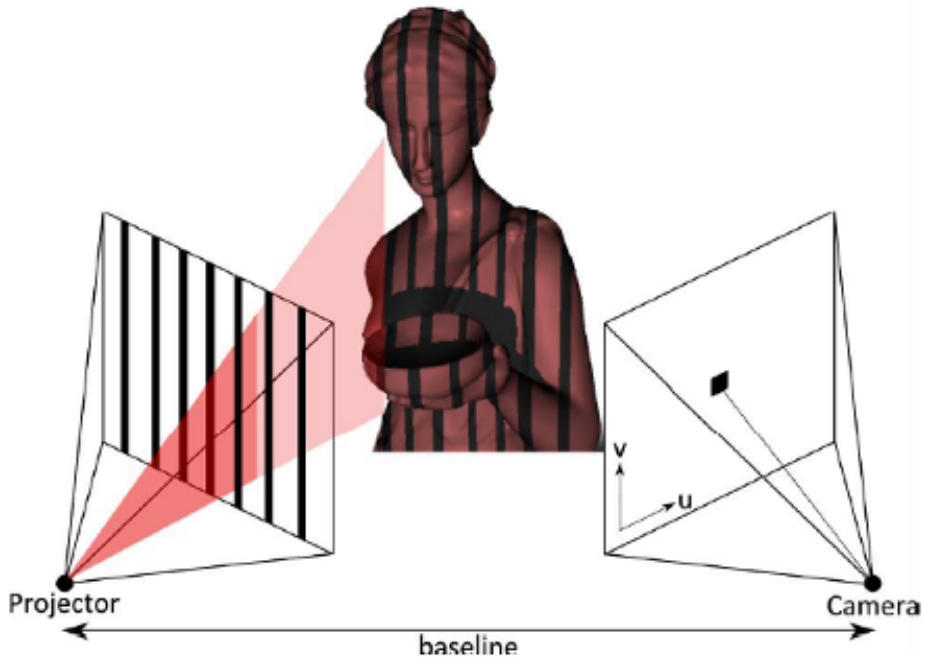
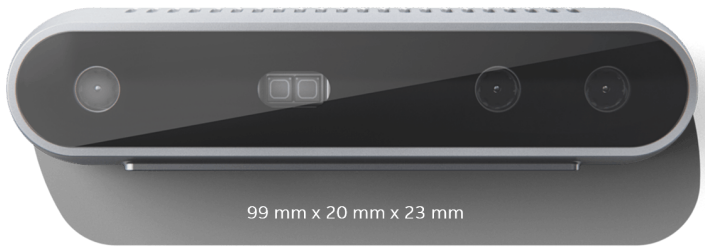


Figure 1: Principle of structured light based systems.

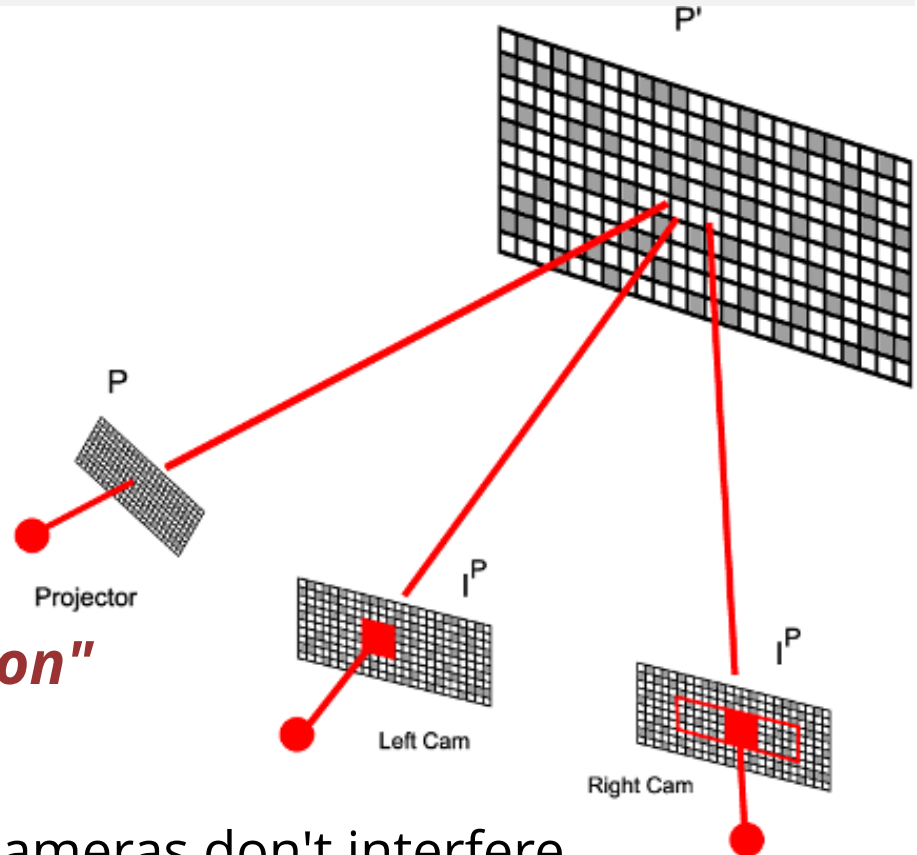
https://arxiv.org/abs/1505.05459

Geometry sensors -- projected texture stereo

(also iPhone TrueDepth)



Intel Realsense D415



Our pick for the "Manipulation Station"

Major advantage over e.g. ToF: multiple cameras don't interfere with each other.

RGB only

https://www.matthewtancik.com/nerf



to the board...

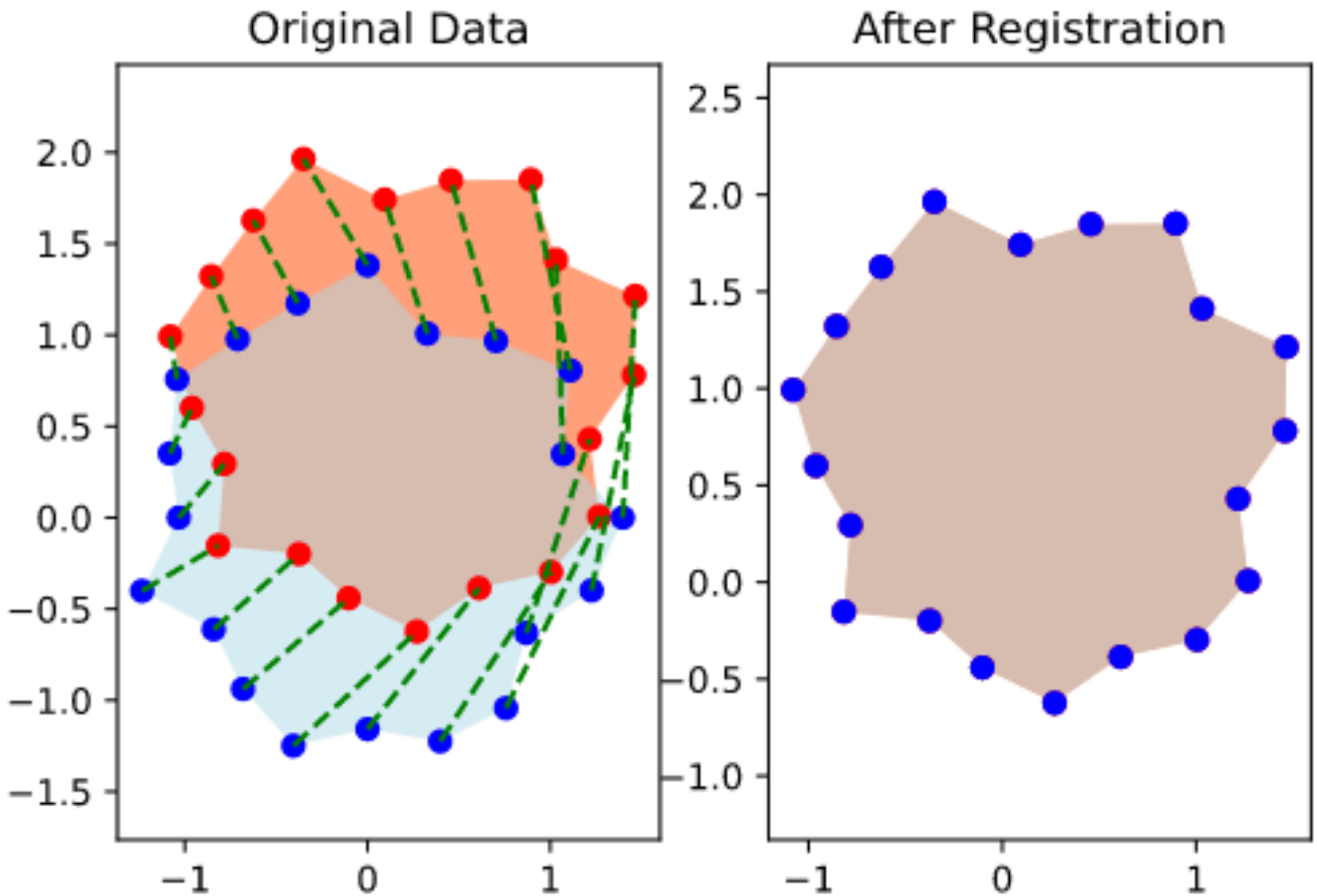
Solving for alignment transform

$$\min_{R \in \Omega, d} \sum_{i=1}^{n} \| Rx_i + d - y_i \|^2,$$

where

$$\Omega = \{R \,|\, R^T R = R R^T = I_3; det(R) = 1\},$$

1. $\overline{x} = \frac{1}{n} \sum_{i=1}^{n} x_i, \quad \overline{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$
2. $A = [x_1 - \overline{x}, \ldots, x_n - \overline{x}], \quad B = [y_1 - \overline{y}, \ldots, y_n - \overline{y}]$
3. $C = BA^T$
4. $U\,S\,V^T = C$ *(Computation of the singular decomposition of* C*)*
5. $R = U\, diag(1, 1, det(UV^T))\, V^T$
6. $d = \overline{y} - R\overline{x}$

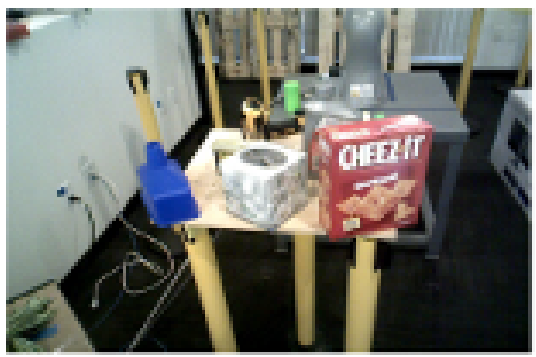

Original Data
After Registration
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1
0
1
−1
0
1

https://manipulation.csail.mit.edu/data/iterative_closest_point.html?height=450px

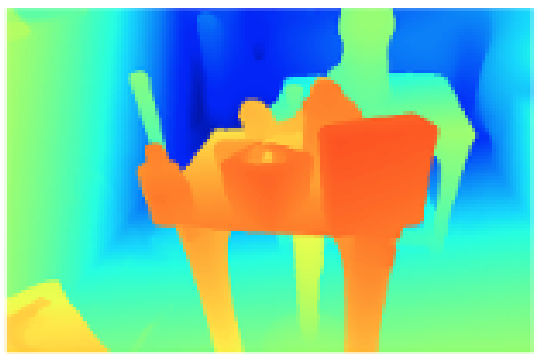
https://www.youtube.com/embed/uzOCS_gdZuM?mute=1&enablejsapi=1&controls=0&rel=0

Real point clouds are also messy

RGB Data



Reprojected Depth Image



Real Depth Image

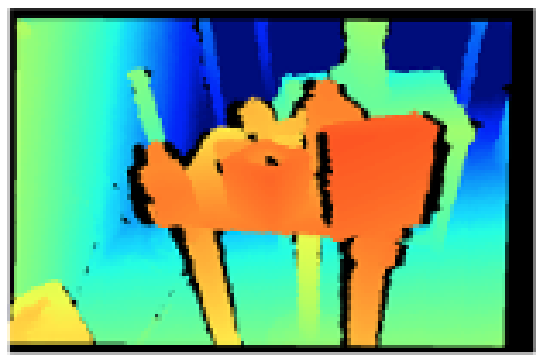
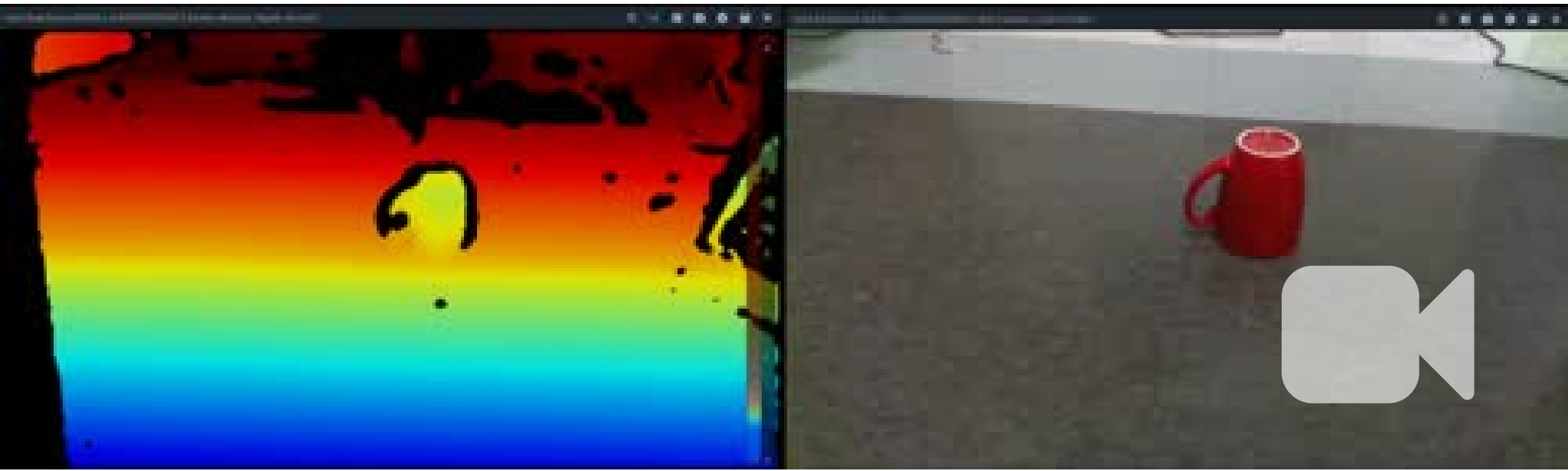


figure from Chris Sweeney et al. ICRA 2019.



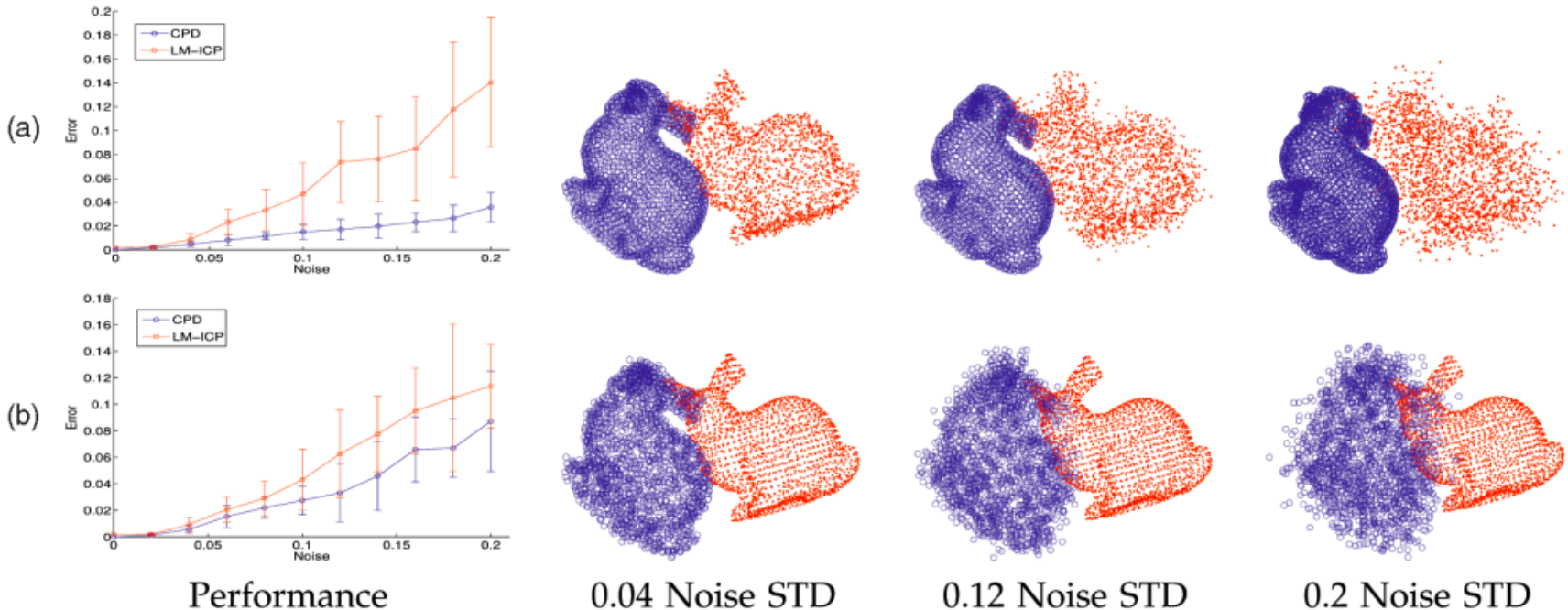
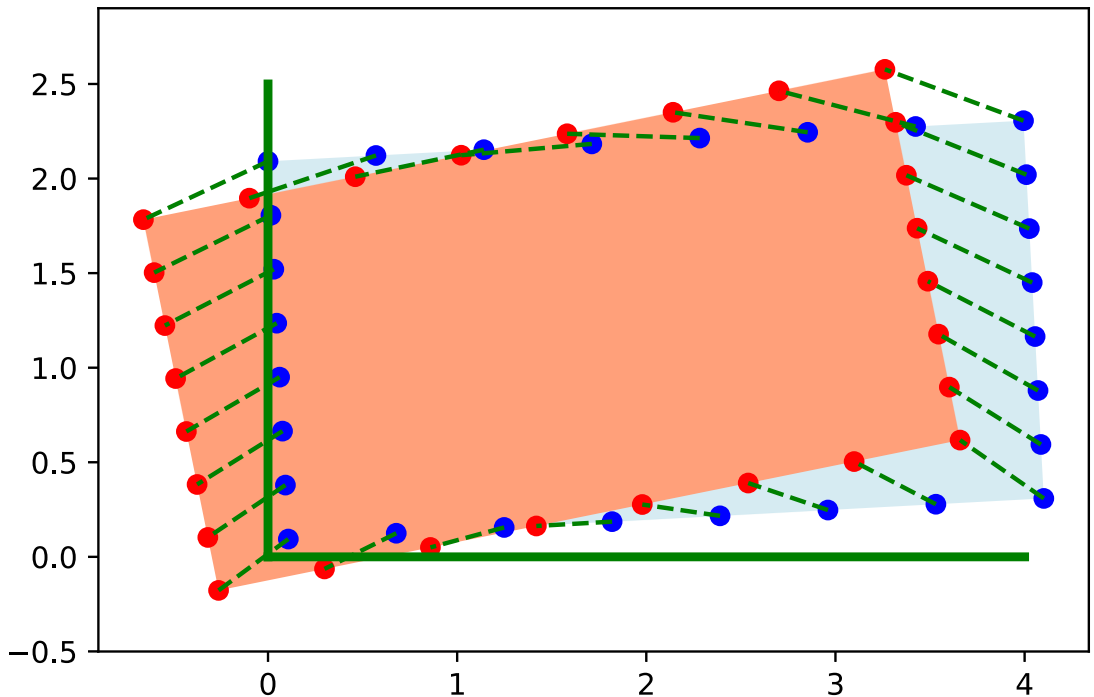


Fig. 8. A comparison of CPD and LM-ICP rigid registration performances with respect to (a) noise in the $\mathbf{X}$ and (b) the $\mathbf{Y}$ point sets. We align $\mathbf{Y}$ (blue circles) onto $\mathbf{X}$ (red dots). Columns 2, 3, and 4 show the examples of initial point sets for different random noise stds added to the point set positions. The first column shows the error in estimating the rotation matrix for CPD (blue) and TPS-RPM (red). CPD outperforms LM-ICP in all cases.


Huber loss
Truncated Least-Squares
Gaussian
Lorentzian
2.5
2
1.5
1
0.5
0
−2.5
−2
−1.5
−1
−0.5
0.5
1
1.5
2
2.5
Distance


2.5
2.0
1.5
1.0
0.5
0.0
−0.5
0
1
2
3
4

```
prog = MathematicalProgram()
# x is a symbolic variable
x = prog.NewContinuousVariables(2)

# numerical, pass in the coefficients:
# x'Qx + b'x + c
prog.AddQuadraticCost(Q, b, c, x)

# symbolic:
prog.AddCost(x.dot(x))
prog.AddQuadraticCost(x.dot(x))

# autodiff:
def my_cost(x):
        return x.dot(x)
prog.AddCost(my_cost, x)

# Adding autodiff costs/constraints => Solve(prog) will
# pick a nonlinear optimization solver
```

```
prog = MathematicalProgram()
p = prog.NewContinuousVariables(2, 'p')
theta = prog.NewContinuousVariables(1, 'theta')

def position_model_in_world(vars, i):
    [p, theta] = np.split(vars, [2])
    R = np.array([[np.cos(theta[0]), -np.sin(theta[0])],
                  [np.sin(theta[0]), np.cos(theta[0])]])
    p_Wmci = p + R @ p_Omc[:,i]
    return p_Wmci

def squared_distance(vars, i):
    p_Wmci = position_model_in_world(vars, i)
    err = p_Wmci - p_s[:,i]
    return err.dot(err)

for i in range(Ns):
    # forall i, |p + R*p_Omi - p_si|²
    prog.AddCost(partial(squared_distance, i=i),
                 np.concatenate([p[:], theta]))
    # forall i, p + R*p_Omi >= 0.
    prog.AddConstraint(partial(position_model_in_world, i=i),
                       vars=np.concatenate([p[:], theta]),
                       lb=[0, 0], ub=[np.inf, np.inf])

result = Solve(prog)
```